

ESFA: An Efficient Scalable FFT Design Framework on Versal AI Engine

Hao Yang , *Student Member, IEEE*, Linfeng Du , *Student Member, IEEE*, Shangkun Li ,
and Wei Zhang , *Senior Member, IEEE*

Abstract—The fast fourier transform (FFT) is widely used to convert a time-domain signal into its frequency-domain representation in various fields. Previous works have demonstrated efficient FFT implementation on various accelerators. The emergence of AI Engines (AIE) on AMD Xilinx’s Versal ACAP brings the possibility of further improvement in computing efficiency. However, previous solutions have been restricted to a single-AIE manner, which limits the FFT size and neglects the potential of employing multiple AIEs. This paper proposes the ESFA framework, which can efficiently and automatically implement a scalable FFT on the Versal ACAP with multiple AIEs. The framework includes an analytical model to report the quality of results (QoRs) estimation for legal FFT partition modes, comprehensively covering the throughput-resource trade-off choices across the design space. In addition, the layout problem is formulated in an ILP to enhance the area efficiency. The framework also incorporates an automatic code generator to enable an agile implementation of the desired design. Our experiments on the VCK190 board show that we achieve 9,226/2,059 *MS/s* simulation/system throughput on the 1K-point FFT with a data width of 32, which obtains up to 10.1x speedup compared with AMD Xilinx’s library targeting AIE, meanwhile, 17.5x, 23.2x, and 0.9x speedup compared to the state-of-the-art designs on ASIC, CGRA, FPGA.

Index Terms—Versal ACAP, AI engines (AIE), field-programmable gate array (FPGA), fast fourier transform (FFT), partitioning, layout.

I. INTRODUCTION

IN the field of digital signal processing (DSP), the discrete Fourier transform (DFT) performs an essential role in designing, analyzing, and implementing many DSP algorithms and systems [1]. However, the computation complexity of DFT is very high ($O(N^2)$) [2]. Thus FFT [3] is instead used to compute the N -point DFT, which can effectively reduce the computational complexity of the DFT to $O(N \log(N))$ by disassembling the DFT computation for a sequence of length N into progressively smaller DFTs. Given the periodicity and symmetry attributes that are intrinsic to complex twiddle factors [4], FPGA is a promising option for effectively accelerating the FFT algorithm.

This work was supported in part by the National Key R&D Program of China under Grants 2023YFB4405100 and 2023YFB4405103, and in part by the Hong Kong Research Grants Council Collaborative Research Fund under Grant C5032-23G. (Corresponding author: Wei Zhang, Linfeng Du.)

Hao Yang and Wei Zhang are with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, and also with the Guangzhou HKUST Fok Ying Tung Research Institute, Guangzhou, China (e-mail: hyangco@connect.ust.hk; eeweiz@ust.hk). Linfeng Du and Shangkun Li are with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, China (e-mail: linfeng.du@connect.ust.hk; shangkun.li@connect.ust.hk).

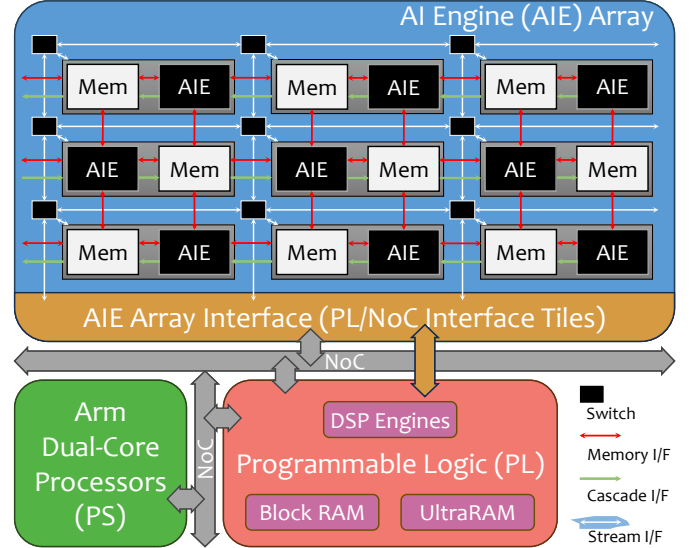


Fig. 1: AMD Xilinx Versal ACAP Architecture.

Figure 1 depicts the major compute units in the comprehensive architecture of the VCK190 board, emphasizing the AIE array. VCK190 is equipped with the first-generation AIE architecture, which consists of 400 7-way VLIW processors running at 1GHz. The VLIW design employed in AI Engines enables the execution of a maximum of two load instructions, one store instruction, one scalar operation, one fixed-point or floating-point vector operation, and two move instructions within a single cycle. Besides, ARM processors are employed for executing Linux and general-purpose applications, and programmable logic (PL) is utilized for designing application-specific hardware, complemented by Digital Signal Processors (DSP). The AI Engine cores and ARM CPUs can be programmed using C/C++ code, while PL can be programmed using both RTL and C/C++ code through the employment of high-level synthesis (HLS). A network-on-chip (NoC) integrates these components within a heterogeneous System-on-Chip (SoC) with I/O peripherals like PCIe and DRAM controllers.

The AI Engines [5] are organized in a two-dimensional array topology, where each engine establishes connections with its adjacent engines in both dimensions. Each AI Engine possesses 16KB of program memory and 32KB of local data memory. This local memory is positioned either to the tile’s left or right, alternating between rows. In total, an AIE can directly access up to 128 KB of memory by sharing

data with neighboring AIEs in four directions. Additionally, each engine features two 32-bit input streams and two 32-bit output streams. These streams are combined with a First-In-First-Out (FIFO) buffer to facilitate convenient 128-bit access every four clock cycles. In the context of data communication among AIEs, three primary connection types are identified: shared memory, AXI Stream, and cascade stream. Data can be exchanged between neighboring AIEs via shared memory using a ping-pong buffer approach, as previously described. Beyond the local memory shared with adjacent tiles, each AIE tile is also linked to an AXI-Stream (AXIS) switch network, facilitating non-local communication between AIE processors. Furthermore, the cascade connection offers 384-bit fine-grained stream access, enabling data transfer between accumulator registers of neighboring AIEs. Unlike shared memory, the cascade stream operates as a unidirectional connection that alternates between rows.

Within the Versal architecture, these three components shown in Figure 1 are integrated with I/O peripherals, such as PCIe and DRAM controllers, to form a heterogeneous SoC characterized by a NoC design. The VCK190 board features 39 array interface tiles that establish communication between AIEs and PL through the programmable logic input/outputs (PLIOs). Collectively, this configuration provides a bandwidth of 1.2 TB/s for PL to AIEs and 0.9 TB/s for AIEs to PL. Additionally, the VCK190 board is equipped with a single DDR4-DIMM off-chip memory, achieving a peak bandwidth of 25.6 GB/s for PL to DDR4 communication, which is significantly lower than the bandwidth available between AIEs and PL.

A. Motivation

However, even though AI Engines can provide substantial computing power, we still need to address several challenges to accelerate the FFT algorithm efficiently. Firstly, it is non-trivial to fully utilize every single AI Engine for computing FFT of different sizes. This necessitates manual **vectorization** of the kernel code for AIEs to balance the data loading, computation, and data rearrangement to obtain the best performance. Besides, an efficient **partitioning approach** is needed to explore the potential of employing multiple AIEs to further improve the performance and scalability. Also when partitioning the FFT across multiple AI Engines, **co-optimization** on workload partitioning to handle the kernel imbalance and dataflow arrangement to decrease the data transfer latency is necessary to maximize the system throughput. Further, an **analytical model** is needed for finding the optimized partitioning configurations under throughput and resource constraints. Moreover, for memory-constrained tasks such as FFT, the AIE Compiler exhibits a high failure rate and significant time consumption when the search space is extensive. Therefore, a **layout scheme** is necessary to facilitate rapid layout generation and to assist the AIE Compiler in identifying an executable solution. Additionally, to reduce the programming efforts for mapping the FFT to the Versal devices, we need an **automatic code generator** to facilitate the system implementation.

B. Contributions

Figure 2 shows the ESFA framework, which serves as a systematic guide for the design of efficient FFT on the Versal architecture. Upon receiving the requisite user inputs, the analytical model integrated within the framework identifies all feasible design options that meet the specified performance and resource requirements. The user can subsequently conduct a trade-off analysis to select the most suitable design configuration. Ultimately, the layout scheme generates the optimized location constraints and the code generator component of the ESFA framework produces the source code for the selected FFT design, targeting the ACAP architecture. The contributions of our work can be summarized as follows:

- exploring techniques for structuring AIE kernels to achieve optimal performance of base size FFT on a single AI Engine,
- a multi-level approach to partition and map FFT to multiple AI Engines to further improve the performance,
- a layout scheme for the rapid generation of location constraints for memory-constraint tasks,
- an analytical model guiding the FFT partitioning choices, along with an automatic code generator facilitating the system implementation,
- obtaining up to 17.5x, 23.2x, and 0.9x speedup compared to the state-of-the-art designs based on ASIC, CGRA, and FPGA.

II. RELATED WORK

The applications of FFT are extensive and varied, encompassing fields such as Digital Signal Processing (DSP), communications, and image processing. Notably, spectral analysis of signals and speech processing represent two fundamental applications of FFT within the realm of DSP. Furthermore, FFT serves as a critical component in contemporary communication systems, particularly in Orthogonal Frequency Division Multiplexing (OFDM) applications, including Digital Broadcasting [6] and Worldwide Interoperability for Microwave Access (WiMAX) [7]. Additionally, FFT-based Long Integer Multiplication is widely utilized in public-key cryptography algorithms, such as RSA (Rivest, Shamir, and Adelman) [8].

Numerous prior studies have leveraged the unique characteristics of computing devices to propose dedicated implementations and parallelism aimed at achieving optimal performance. Specifically, FFT computations have been deployed on various hardware platforms, including CPU, GPU, FPGA, ASIC, and CGRA. While CPU and GPU implementations are typically optimized for algorithms and memory access, they often lack sufficient efficiency. In contrast, FFT implementations on FPGA, ASIC, and CGRA can be tailored with specific hardware units to enhance parallelism. This section introduces FFT implementations across different platforms and presents various designs based on AMD Versal ACAP.

A. FFT on Various Platforms

Most implementations of FFT on the CPU or GPU utilize graphics APIs such as CUDA or DirectX. Nvidia's *cuFFT* [9]

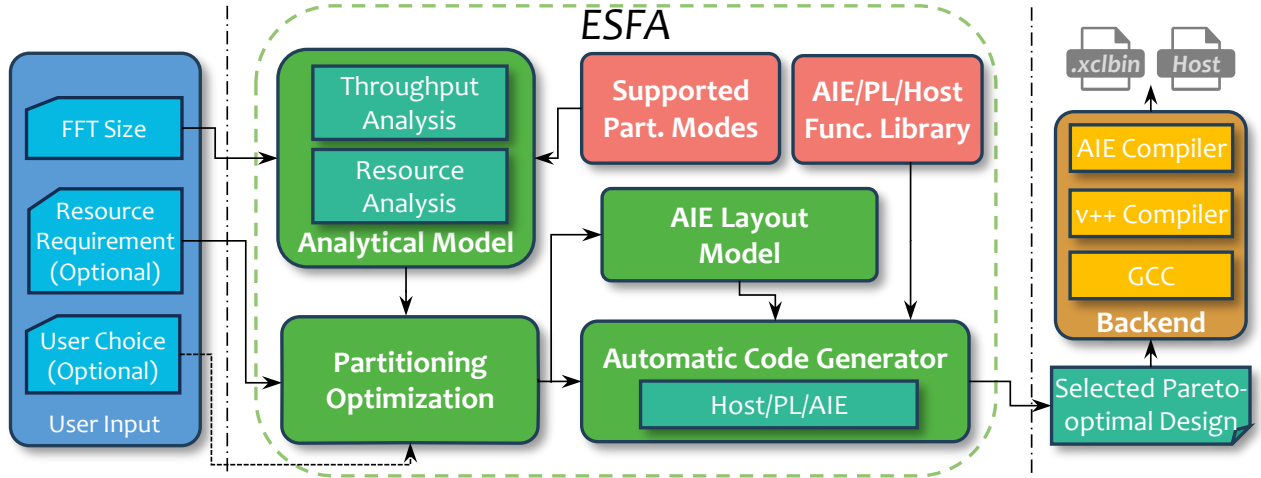


Fig. 2: An Overview of Our ESFA Framework, Exhibiting the End-to-end Flow to Optimize and Implement the FFT Task on the Versal ACAP Platform.

library provides expertly optimized Cooley-Tukey FFT implementations for numerous radix combinations using CUDA. AMD's *clFFT* [10] library generates OpenCL kernels for specific sizes. Prior research on FFT for GPUs [11], [12], [13] presents FFT implementations based on the Stockham algorithm, expressed in CUDA or DirectX9, emphasizing memory access optimization and arithmetic operation minimization in a device-specific manner. Although FFTs are dedicatedly optimized on GPUs, the computational efficiency remains low; for example, that of 1M-point FFT on IBM QS20 and NVIDIA GTX280 is only 9.3% and 6.4%, respectively [14].

In the realm of hardware-based approaches, the implementation of FFT on FPGAs or ASICs has demonstrated superior performance and energy efficiency. This advantage stems from efficiently customizing the computation mechanism and memory organization to align with the parallelism characteristics and data access patterns inherent in FFT algorithms. Consequently, the utilization of FPGA-based FFT accelerators has emerged as a promising avenue for enhancing FFT performance, particularly with advancements in FPGA technology [15].

Prior research has endeavored to improve parallel FFT processing through innovative architectural designs. Nguyen et al. [16] introduced a feed-forward and memory-based iterative architecture to enhance the utilization of arithmetic resources and reduce memory requirements. However, this approach suffered from intricate control logic and large latency. Jian et al. [17] propose a mixed-decimation multipath delay feedback strategy to address the issue of idle time in arithmetic modules in feedback architectures. This approach can potentially reduce the excessive utilization of arithmetic resources while maintaining a relatively high throughput. Another proposal by Kavitha [18] combined improved CORDIC and Radix2^r to reduce the number of adders. This design employed fewer complex adders and multipliers but encoun-

tered challenges such as lower throughput, increased latency, and greater memory demands. Accordingly, higher-radix FFT algorithms [19], [20], [21] can be used to optimize resource utilization. However, the adoption of these algorithms introduces heightened design complexity [2]. Zhang et al. [22] also proposed the MPR-FFT algorithm to reduce the computational resources to a large extent but demands greater memory. Despite these efforts, the performance of FPGA-based FFT implementations still falls short of meeting the demands of DSP applications due to the limitations imposed by low clock frequencies [23].

The ASIC-based FFT offers higher clock frequency and lower power, enabling further throughput and energy efficiency enhancements to meet the real-time DSP requirements. Shih et al. [24] propose a flexible and reconfigurable changeable-radix architecture to support FFT up to 4K size. Additionally, [25] presents a pipelined FFT processor featuring two independent data streams to facilitate the computation of multiple independent FFT operations. Furthermore, several variable-size FFT processors are introduced in [26] to cater to different communication protocol standards. However, ASICs are characterized by their high cost, inflexibility, and limited usage in DSP systems. Consequently, the Coarse-Grained Reconfigurable Array (CGRA) has emerged as an alternative solution [27], [28], [29] due to its ability to strike a balance between flexibility and high performance. Nevertheless, the scalability of the FFT design on CGRA is severely restricted due to the frequent data transfer and the fine granularity of processing elements (PEs).

B. Various Designs on Versal ACAP

Recent research has investigated the capabilities of the Versal ACAP Platform. XVDPU [30] is the first CNN accelerator developed for the Versal platform, designed to address 3D convolution tasks in disparity estimation while achieving high

end-to-end performance. The CHARM framework [31] proposes a method for segmenting and mapping matrix multiplication onto the Versal platform. Subsequent work, AIM [32], extends this framework to support arbitrary-precision integer multiplication. MaxEVA [33] focuses on efficiently mapping matrix multiplication workloads onto the AI Engine, outperforming CHARM. Additional studies have explored the application of the Versal ACAP in stencil computation [34], [35] and market risk analysis [36]. WideSA [37] proposes a mapping scheme for uniform recurrences and generates a systolic design on the ACAP; however, it is not scalable for 2D FFT designs and does not address the layout problem. EA4RCA [38] implements a dedicated FFT processing unit design that supports a limited range of FFT sizes and lacks flexibility. The efficient and scalable implementation of FFT on the Versal ACAP platform, while considering layout constraints, remains an unresolved challenge.

III. ALGORITHM ANALYSIS AND REVISION

In this section, we first introduce the Cooley-Tukey FFT algorithm. Subsequently, we present and extend the single-level partition algorithm to implement FFT across multiple AIEs. Furthermore, we adapt this algorithm to a multi-level structure to enhance both performance and scalability.

The direct implementation of DFT necessitates N^2 complex multiply-and-accumulate (MAC) operations, which can be significantly reduced to $N \log_2 N$ through the employment of the Cooley-Tukey FFT algorithm. Assuming an input of length $N = 2^m$, an FFT operation takes time-domain samples $\{x_0, x_1, \dots, x_{N-1}\}$ and produces a frequency-domain outputs on subcarriers $\{X_0, X_1, \dots, X_{N-1}\}$, formulated as follows:

$$X(K) = \sum_{r=0}^{\frac{N}{2}-1} x(2r)W_N^{2rk} + \sum_{r=0}^{\frac{N}{2}-1} x(2r+1)W_N^{(2r+1)k} \quad (1)$$

where the twiddle factor W_N^{2rk} is calculated as $e^{-j\frac{4\pi rk}{N}}$. The Cooley-Tukey FFT algorithm is extensively utilized in various research studies and commercial IPs. In practice, the Xilinx DSP Library offers small-point FFT/IFFT IP on a single AI Engine with limited optimization. Additionally, the library supports a maximum size of 2K due to memory constraints. Consequently, a multiple AI Engine implementation is necessary for improved efficiency and scalability. To facilitate this implementation, developing a partitioning algorithm that divides the entire FFT task into smaller components is essential, thereby reducing the workload on a single AI Engine and enhancing the overall performance.

A. Single-level Partition

Building upon [39], we propose partitioning a complete FFT into multiple levels by employing searched partition factors. The partitions are then distributed across multiple AIEs to establish a pipelined data flow. In the context of an FFT of size N , with partition level 1, partition factor F , and the size

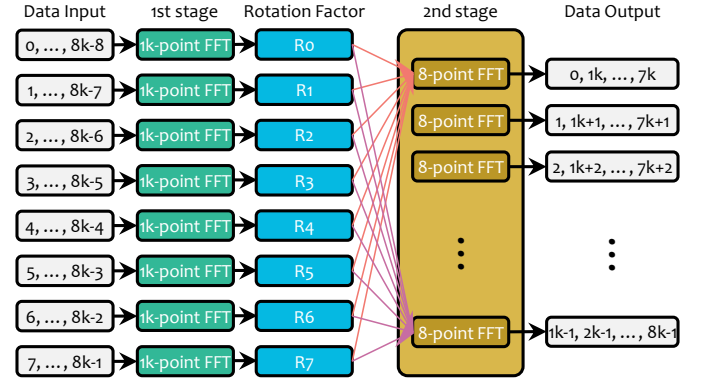


Fig. 3: The Data Flow of the Single-level Partition $8K = 8 \times 1K$ with Partition Level 1 and Partition Factor 8.

of the base FFT B ($B = N/F$), we introduce the formulation of the single-level partitioning algorithm as follows:

$$\begin{aligned} X(lB + m) &= \sum_{i=0}^{F-1} \sum_{q=0}^{B-1} x_{\{qF+i\}} e^{-j \frac{2\pi(lB+m)(qF+i)}{FB}} \\ &= \sum_{i=0}^{F-1} e^{-j \frac{2\pi i m}{FB}} \sum_{q=0}^{B-1} (x_{\{qF+i\}} e^{-j \frac{2\pi q m}{B}}) e^{-j \frac{2\pi i l}{F}} \end{aligned} \quad (2)$$

For the $\sum$ notation in the middle, we can extract a B-point FFT with the form:

$$X(m) = \sum_{q=0}^{B-1} x_{\{qF+i\}} e^{-j \frac{2\pi q m}{B}} \quad (3)$$

Subsequently, by multiplying $X(m)$ with the linear phase rotation factor $e^{-j \frac{2\pi i m}{FB}}$ and denoting the result as $X'(m)$, the external summation finally yields an F -point FFT with the form:

$$X(l) = \sum_{i=0}^{F-1} X'(m) e^{-j \frac{2\pi i l}{F}} \quad (4)$$

By employing this partition algorithm, we can divide the computation of an N -point FFT into two distinct steps, where $N = F \times B$. Initially, we perform B -point FFT operations F times on the corresponding inputs to obtain $X(m)$. Subsequently, utilizing the intermediate results, F -point FFT computations are executed B times to derive the final outputs ($x(l)$).

For instance, Figure 3 illustrates the data flow of the 8K-point FFT partitioned to $8 \times 1K$, which means the partition level is 1, the partition factor F is 8, and the base size B is 1K. As shown in the figure, the data inputs are separated and reorganized based on the partition factor and base size. In the first stage, eight 1K-point FFTs are executed in parallel with their corresponding inputs. The intermediate results from the first stage are then multiplied by the linear rotation factor $R_p, p \in \{0, 1, \dots, 7\}$. Subsequently, the eight 1K outputs, after applying the linear rotation factor, are evenly distributed to 1K 8-point FFTs in sequential order (e.g., the first 8-point FFT takes every first element of the eight 1K outputs), where

the second stage computation is conducted to obtain the final outputs.

B. Multi-level Partition

Following the partitioning approach in single-level partition, we can further iteratively partition the base FFT to develop a multi-level partitioning algorithm. The algorithm consists of two primary components. The initial data preprocessing phase organizes the input data for each base FFT kernel according to the FFT size, partition level, and partition factors. In the subsequent computation phase, FFT kernel computation and linear rotation factor multiplication are performed at each level. Data transfer between different levels is determined by the corresponding level number and partition factor. The ESFA framework automatically manages all these configurations.

As illustrated in Alg. 1, we utilize multi-level partitioning to decompose an FFT of size N to L levels, where the partition factors at each level are defined as $F_k, k \in \{1, 2, \dots, L\}$. At the last (L_{th}) level, the kernel responsible for the B -point FFT computation is designated as the base FFT. Consequently, the multi-level partitioning can be expressed as $N = F_1 \times \dots \times F_k \times B$, where $F_k, k \in \{1, 2, \dots, L\}$ denotes the partition factor at each level and B is the size of the base FFT.

Contrary to the top-down formulation process in section III-A, the computation process in Alg. 1 adopts a bottom-up approach, beginning with the base FFT. Prior to the computation phase, data preprocessing is necessary to rearrange the input for each base FFT kernel. According to the multi-level partition algorithm, there are F_s base FFT kernels, where $F_s = F_1 \times \dots \times F_k$, and each kernel performs a B -point FFT. Given the original input sequence $\{x_0, x_1, \dots, x_{N-1}\}$, the input data for the i_{th} base FFT kernel is reorganized as $[x_i, x_{i+F_s}, \dots, x_{i+N-F_s}]$ (line 4).

During the computation phase, firstly, the F_s base FFT kernels perform B -point FFT computations based on the corresponding input (line 11). The intermediate results are then stored in the *temp* buffer and multiplied by the rotation factor R_i (line 12). The subsequent L levels integrate the intermediate results level-by-level to produce the final outcomes (lines 15-38). At level l , there are total F_s/F_l F_l -point FFT kernels (line 17), and the corresponding input data are rearranged based on the kernel number q and partition factor F_l (lines 18-20). After executing the F_l -point FFT kernels (line 21), the intermediate results are segmented and stored in the temporal buffers in the same order as the input data (lines 22-24). And if l is not the last level (line 26), the intermediate results are multiplied by the rotation factor R'_i , where N' represents the FFT size after the l_{th} level's integration (lines 27-28). Specifically, there are F_s/F_{l-1} F_{l-1} -point FFT kernels (line 29), with each kernel having F_{l-1} types of rotation factors (line 30). Eventually, the final results of the N -point FFT are obtained after integrating across L levels (line 36). Utilizing this multi-level partition algorithm allows for the further division of the entire FFT task into multiple components, thereby facilitating the use of extra AIEs to enhance performance. Furthermore, the various combinations of partition levels and partition factors can adapt flexibly to diverse requirements, demonstrating superior scalability compared to dedicated designs.

Algorithm 1: Multi-level FFT Partitioning Algorithm

Input: FFT size N , input data x , number of levels L , partition factor F_k with $k \in \{1, 2, \dots, L\}$, base size B

Result: N -point FFT outputs

```

1 Data Pre-processing;
2 Denote  $F_s = F_1 \times \dots \times F_k$ ;
3 for  $i \in \{0, \dots, F_s - 1\}$  do
4   | Data input  $D_i \leftarrow [x_i, x_{i+F_s}, \dots, x_{i+N-F_s}]$ 
5 end
6 Computation;
7  $comp\_cnt = 0$ ;
8 rotation factor  $R_i = e^{-j\frac{2\pi im}{N}}$ ,  $i \in [0 : F_s - 1]$ ,
    $0 \leq m \leq B - 1$ ;
9 while  $comp\_cnt < F_s$  do
10  |  $i \leftarrow comp\_cnt$ ;
11  |  $temp \leftarrow B\text{-point FFT}(D_i)$ ;
12  |  $temp_i \leftarrow temp \times R_i$ ;
13  |  $comp\_cnt++$ ;
14 end
15 for  $l \in \{L, L-1, \dots, 1\}$  do
16  | denote  $F' = F_L \times \dots \times F_l$ ;
17  | for  $j \in \{0, \dots, F_s/F_l - 1\}$  do
18  |   | for  $q \in \{0, \dots, F_l - 1\}$  do
19  |   |   |  $data.append \leftarrow temp_{j+q \times F_s/F'}$ ;
20  |   |   end
21  |   |  $interm[:] \leftarrow F_l\text{-point FFT}(data)$ ;
22  |   | for  $q \in \{0, \dots, F_l - 1\}$  do
23  |   |   |  $temp_{j+q \times F_s/F'} \leftarrow split(interim[:])$ ;
24  |   |   end
25  |   end
26  | if  $l \neq 1$  then
27  |   | denote  $F' = F_L \times \dots \times F_{l-1}$ ,  $N' = B \times F_L \times \dots \times F_l$ ;
28  |   | rotation factor  $R'_i = e^{-j\frac{2\pi im}{N'}}$ ;
29  |   | for  $i \in \{0, \dots, F_s/F_{l-1} - 1\}$  do
30  |   |   | for  $t \in \{0, \dots, F_{l-1} - 1\}$  do
31  |   |   |   |  $temp_{i+t \times F_s/F'} \leftarrow temp_{i+t \times F_s/F'} \times R'_i$ ;
32  |   |   |   end
33  |   |   end
34  |   end
35  | else
36  |   |  $return interm[:]$ ;
37  | end
38 end

```

IV. IMPLEMENTATION

A. Single Kernel

Achieving high efficiency in single-AIE computation requires a manually vectorized kernel that fully exploits the inherent 7-way VLIW capability of AIE. This necessitates a balance among data loading, calculations, and data rearrangement to achieve the highest performance. By pipelining the calculations and removing the data dependencies, we can obtain the minimum critical cycles of the AIE kernel. Therefore, we propose a coding strategy to optimize the kernel efficiency of AIE, as shown in Listing 1. Following the partition algorithm, the size of the FFT should be a power of 2 (i.e., $N = 2^q$). Thus, we constructed radix-2 and radix-4 FFT functions as the base components to compose the base FFT kernel in a single AIE. Then, in the ESFA framework, FFT of different sizes can be represented as $N = 2^r \times 4^s$ while utilizing the optimized radix-2 and radix-4 FFT functions to obtain enhanced performance.

```

1 bufa = upd_v(bufa, 0, *px); px += 64;
2 bufa = upd_v(bufa, 1, *px); px += 64;
3 bufa = upd_v(bufa, 3, *px); px += 64;
4 bufa = upd_v(bufa, 2, *px); px -= 191;
5 for (unsigned int j = 0; j < (1024 / 32) - 1; ++j)
6     chess_prepare_for_pipelining
7     chess_loop_range(4, ) {
8         // computation, data rearrange, and bufb update
9         *py = lsrs(mul4_sym(bufa, ..., 0), 0); py +=
10         64; bufb = upd_v(bufb, 0, *px); px += 64;
11         *py = lsrs(mul4_antisym(bufa, ..., 2), 0); py +=
12         64; bufb = upd_v(bufb, 1, *px); px += 64;
13         *py = lsrs(mul4_sym(bufa, ..., 1), 0); py +=
14         64; bufb = upd_v(bufb, 3, *px); px += 64;
15         *py = lsrs(mul4_antisym(bufa, ..., 3), 0); py -=
16         191; bufb = upd_v(bufb, 2, *px); px -= 191;
17         // computation, data rearrange, and bufa update
18         ... // 4 similar lines exchanging bufb with bufa
19     }

```

Listing 1: An Optimized Procedure for Radix-4 FFT Function Utilizing the Coding Strategy.

Listing 1 demonstrates the overall pipelined data processing and calculation of the radix-4 FFT function. Before the loop body, we use the pointer px of the input data to rearrange the input sequence and update the first group of vectorized data to $bufa$. Similarly, the output data sequence is efficiently controlled by pointer jumps. Then, in the loop, $bufb$ serves as the second buffer to store the next group of data while the computation is ongoing. $bufa$ and $bufb$ serve as ping-pong buffers to remove data dependency and hide the data's loading time within the computation process. The vectorized computation function can process multiple calculations in one cycle and alternatively arrange with data loading. This coding strategy tiles data into ping-pong buffers to eliminate data dependency and inserts vectorized computation functions alternatively to achieve a fully pipelined calculation process. Through this core procedure, the simulated critical cycle is the minimum one constrained by the resources. The minimized time consumption of the single AIE computation demonstrates the high efficiency of this coding strategy.

Due to the memory constraints, one AIE can access up to 128 KB of the local memory without incurring additional data transfer latency and, therefore, support up to 2K-point FFT (data type is `cint16`). However, the 2K-point FFT implementation occupies too much memory to store its parameters. The exhaustive memory usage could result in low AIE utilization, and hence, it is inefficient to serve as the base of multi-level partitions. Moreover, when the FFT size is smaller than 64, the data transfer time from the PL to the AIE array would be larger than the computation time of the AIE, resulting in low efficiency on the AIE. Therefore, ESFA searches the size options among $\{64, 128, 256, 512, 1024\}$ for single-AIE implementation.

B. Single-level Partition

To explore the potential of employing multiple AIEs and achieving FFT with larger sizes, we implement the single-level partition algorithm to divide the FFT into several components and allocate them to adjacent AIEs. After the partition, each stage has a shorter period due to less computation intensity, and different stages are pipelined to enhance the performance.

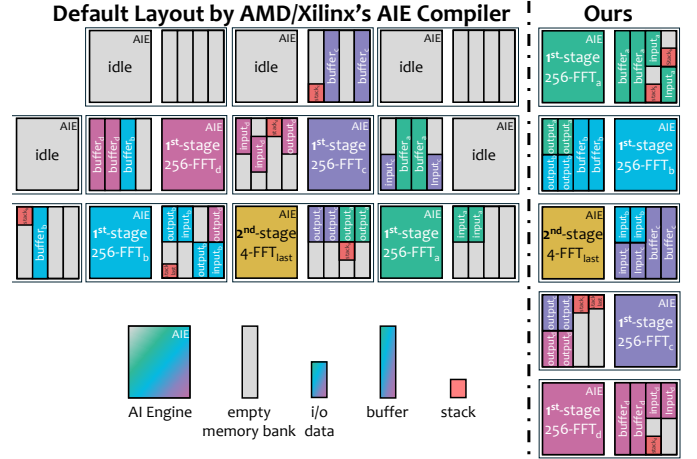


Fig. 4: Default vs. Our Optimized Layout for Single-level Partition $1K = 4 \times 256$.

Consequently, the system throughput can be much higher than the single-kernel designs.

Corresponding to the form of the FFT size ($N = 2^q$), the partition factor F should also be a power of 2, e.g., 2, 4, 8, 16, ... And for the kernels on the AIE, each call requires some time to restart and initialize them. Therefore, for the base FFT smaller than 1K, several data input is grouped to form 1K inputs to avoid frequent and continuous calls of the same kernel. To remove the transfer latency of intermediate results between stages, the output data from a previous stage should be stored in the shared local memory that the next stage can access directly. For example, consider the single-level partition $1K = 4 \times 256$, where the partition factor is 4 and the base size is 256. The first stage has four AIEs to execute the 256-point FFTs in parallel. The output data generated by these four AIEs should be stored in the shared local memory that the last (second) stage can directly access without incurring additional latency, thereby enhancing the overall throughput. The AIEs connected through shared memory employ a ping-pong buffering method to mitigate communication overhead. During the processing phase, while the second stage's AIE computes data in the ping buffer of the first stage's AIEs, the first stage's AIEs concurrently store the results in their pong buffer. This access pattern alternates with each time step, thereby enhancing efficiency in data handling. The left portion of Figure 4 shows the default AIE and data layout produced by the AIE compiler. Although the last stage (gold AIE) can access intermediate results instantly, the overall layout is sparse and not area-efficient. Manual optimization of the layout is feasible for small-scale designs; however, it becomes complicated and time-consuming for large-scale designs. To address this issue, we propose a layout scheme designed to identify the optimized configuration efficiently. Further details regarding the layout scheme are provided in Section IV-D. As illustrated in the right section of Figure 4, our optimized design meets memory constraints while defragmenting data buffer allocation. The optimized layout is more compact and involves fewer AIEs than the default design. Additionally, it facilitates the convenient and rapid implementation of additional

instances on the AIE array, thereby enhancing AIE utilization. Furthermore, a regular shape is more compatible with other designs which contain FFT operation.

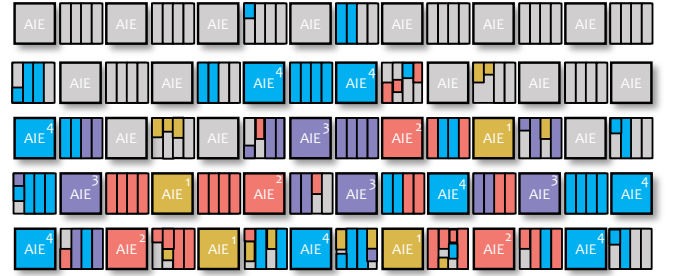
However, when we set the partition factor to 16, the last AIE would require up to 128KB of memory to store the intermediate results and leave no space for other parameters like system stack and heap. Hence, ESFA searches the partition factors among $\{2, 4, 8\}$ for both single-level and multi-level partition designs on the AIE array.

C. Multi-level Partition

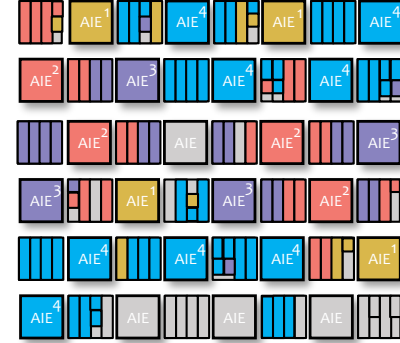
Based on the single-level approach, further partitioning the base FFT is feasible to compose the multi-level partition as $N = F_1 \times F_2 \times \dots \times F_L \times B$, where L is the number of partition levels. Even though the principle is similar to the single-level approach, we still need to address several newly triggered issues. First, each level requires a specific rotation factor set based on the input size and the partition factor, which is time-consuming to fully implement. To solve this problem, we designed a group of optimized and reusable functions by balancing the workload as shown in Section IV-A and constructed corresponding code templates. Second, except for the initial level in a multi-level partition pipeline, all subsequent levels experience performance degradation due to a bottleneck in the data rearrangement step. ESFA systematically considers the FFT size and each subsequent level's partition factor to optimize data rearrangement for the current level. Furthermore, the layout of multi-level partitions can become highly complex when the number of partition levels is large (e.g., $L \geq 4$), and the AIE compiler often fails if location constraints are not provided. To address this issue, we utilize our layout model to identify the optimized layout efficiently, which is then used to guide the AIE compiler in finding a feasible solution. Figure 5 compares the default layout with our optimized layout for a multi-level partition example. The AIEs and memory banks of different levels are colored differently. In comparison to the default layout, the optimized layout utilizes fewer resources, resulting in improved area and energy efficiency. Additionally, by limiting the search space in the layout model, the shape of the generated design can be more regular, which is suitable for duplicating or combining with other modules.

D. Layout Model

For memory-constrained tasks such as FFT, the location of data is critical for ensuring timely access without latency. A stall in memory access can cause delays throughout the entire processing pipeline, leading to performance degradation. Furthermore, the layout of the design significantly affects area and energy efficiency, depending on the AIEs and memory banks utilized. The memory constraints can be formulated as follows: First, each AIE's data (e.g., input, output, temp buffer) should be stored in its own memory or local shared memory banks. Second, for AIEs with data dependencies, the outputs of preceding AIEs should be stored in memory banks that subsequent AIEs can directly access. Since the AXI Stream connection necessitates the copying and pasting



(a) Default Layout by AMD/Xilinx's AIE Compiler



(b) Our Optimized Layout

Fig. 5: Multi-level Partition Layout Comparison for $1K = 2 \times 2 \times 2 \times 128$. (For both AIE Core and Memory Banks: Light blue: level 4/base FFT, Lavender: level 3, Rose: level 2, Yellow: level 1/last level). The Number in the Upper Right Corner Denotes the Level Number.

of required data between AIEs that exhibit data dependencies. This process incurs additional transmission time and requires supplementary memory for data storage, ultimately leading to pipeline stalls and increased memory requirements. In contrast, the shared memory manner allows subsequent AIEs to directly access the data without the need for transfer, thereby mitigating delays and alleviating the associated limitations. If this constraint cannot be fulfilled due to memory limitations, subsequent AIEs should still be able to access part of the required data directly. Additionally, the total resources involved in the layout design should be minimized to enhance area and energy efficiency.

In ESFA, the layout model is formulated as an ILP (Integer Linear Programming) problem utilizing Gurobi. We model the AIE topology as a two-dimensional array and incorporate linear constraints to restrict the data locations of each kernel. Table I contains the necessary symbols in the formulation. As illustrated in the following formulas, we first assign the partitioned tasks to the AIE cores (C1) and the corresponding data to the memory (C2). It is essential to ensure that each task and data item is allocated only once and does not exceed the memory capacity. The third constraint (C3) delineates the restriction of the current level's data for the corner case ($row = 0, col = 0$). Other cases vary in their constraints depending on the location of the AIE core, indicating that the data for the current level should be stored in the shared local

TABLE I: Symbols Utilized in the Layout Model Formulation.

Symbols	Definition
r/c	Number of rows/columns.
n_r/n_c	The maximum number of rows/columns.
t	Number of partitioned components, referred as tasks.
d/d_t	The number/maximum number of data types.
t_n	Number of tasks at the next level.
n_i	Number of inputs for the task t_n .
J	Set of related tasks at the previous level for task t_n .
$T[r, c, t]$	Whether task t is located at r, c .
$D[r, c, t, d]$	Whether data d of task t is located at r, c .
$Y[r, c]$	Whether the AIE at r, c is involved in the design.

memory of its corresponding AIE core. The fourth constraint ($C4$) defines the relationship between different levels, stipulating that the output from the previous level should be stored in the shared local memory of the corresponding AIE core at the next level. Finally, we set the objective (Obj) to find the most area-efficient solution within these constraints. The layout model can be readily modified to adjust the topology size and the connection relationships between AIEs, allowing for adaptation to various tasks. Furthermore, a compact and regular layout can enhance the integration of multiple tasks. When several tasks are assigned, the process of directly inputting them into the layout model may be time-consuming due to the extensive search space involved. Consequently, our layout model utilizes a divide-and-conquer strategy. It entails generating an optimal layout for each individual task and subsequently assembling these regular task blocks into a whole. The overall time and the quantity of AIEs can be represented as a linear aggregation of the results obtained from each individual task.

$$\text{minimize} \quad \sum_{r=0}^{n_r} \sum_{c=0}^{n_c} Y[r, c] \quad (Obj)$$

$$\text{subject to} \quad \sum_{r=0}^{n_r} \sum_{c=0}^{n_c} T[r, c, t] == 1 \quad (C1)$$

$$\sum_{r=0}^{n_r} \sum_{c=0}^{n_c} D[r, c, t, d] == 1 \quad (C2)$$

$$\sum_{d=0}^{d_t} D[r, c, t, d] + D[r+1, c, t, d] == d_t, \\ \text{if } T[r, c, t] == 1, r = c = 0 \quad (C3)$$

$$\sum_{t \in J} D[r, c, t, 3] + D[r+1, c, t, 3] == n_i, \\ \text{if } T[r, c, t_n] == 1, r = c = 0 \quad (C4)$$

E. Analytical Model

To report the quality of results (QoRs) estimation for legal partition modes, we design an analytical model that comprehensively covers the throughput-resource trade-off choices across the design space. It takes the FFT size (N) and optional

TABLE II: Symbols Utilized in the Analytical Model Formulation.

Symbols	Definition
F_i	Partition factor for level i . $F_0 = 1$.
N_s	Number of input samples.
L	Number of partition levels.
f	Working frequency of AIE.
C_i^l, C_j^b	Estimated execution cycles for i_{th} -level and j_{th} -base kernel.
E	Pipeline efficiency between PL and AIE, assigned 70%.
b	Pre-characterized bias factor.

user-specified resource requirements (e.g., the maximum number of AIEs utilized) as input and does an analytical model-based search. Table II contains the necessary symbols in the formulation.

For an L -level partition with the mode $N = F_1 \times \dots \times F_L \times B$, the number of AIEs M needed can be formulated as:

$$M = \prod_{i=0}^L F_i + \sum_{j=0}^{L-1} \frac{\prod_{i=0}^L F_i}{F_j} + M_L \quad (5)$$

M_L represents the number of AIEs of the last level and has the form:

$$M_L = \begin{cases} \frac{\prod_{i=0}^L F_i}{F_L}, & F_L = 2 \text{ or } 4 \\ 4 \frac{\prod_{i=0}^L F_i}{F_L}, & F_L = 8 \end{cases} \quad (6)$$

Among all estimated execution cycles of level/base kernels $C^{All} = \{C_0^l, \dots, C_L^l, C_0^b, \dots, C_{\prod_{i=0}^L F_i}^b\}$, we take the maximum to calculate the execution time and get the throughput P :

$$P = \frac{N_s f E}{\max\{C^{All}\} + b} \quad (7)$$

Upon identifying the optimal mode, we utilize our automatic code generator to generate the corresponding source code for the AIE, PL, and host CPU based on the optimized templates. Following code generation, the ESFA launches the AIE compiler and the V++ compiler in the vendor tools to generate and link the output object files into a single hardware bitstream file (*.xclbin*). In addition, the GCC compiler compiles the host code to run on the ARM CPU for scheduling and system control. Based on the components discussed in Section IV, ESFA framework enables an agile end-to-end implementation of the desired design as depicted in Figure 2.

V. EXPERIMENT RESULTS

In this section, we commence with an analysis of the single AIE efficiency and the throughput associated with both single-level and multi-level partition designs, as discussed in Sections V-A, V-B, and V-C. Subsequently, in Section V-D, we conduct a comparative power analysis between our design and the FFT implementation accelerated by GPU. Following this, we implement various designs to validate the accuracy of our layout and analytical models presented in Sections V-E through V-F. All experiments were executed on the VCK190

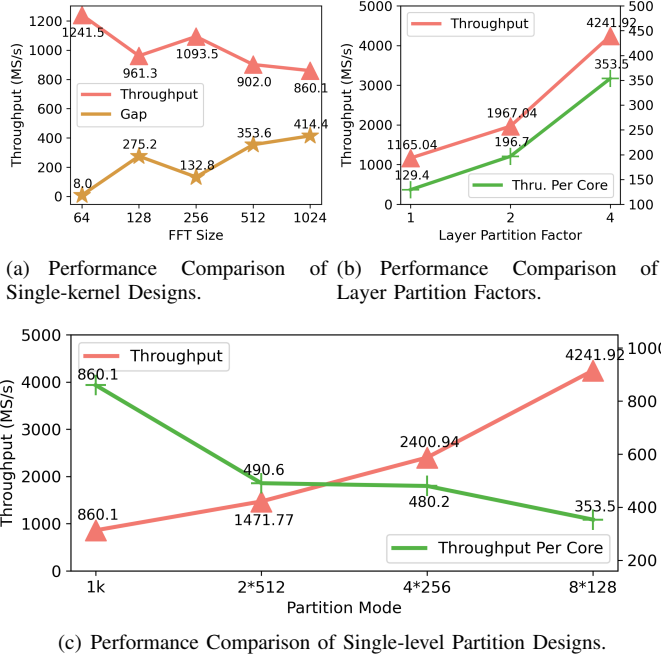


Fig. 6: Throughput and Efficiency Comparison under Different Scenarios.

platform, utilizing AMD/Xilinx Vitis version 2022.2 as the compilation backend tool. In measuring power consumption, we iterated through each application and reported the average values obtained.

A. Single Kernel Comparison

This section showcases our single AIE computation efficiency under different FFT sizes with datatype `cint16`. We obtain the execution output with a time stamp of our single AIE design by simulating it on the Vitis Analyzer. Then, we calculate the throughput based on the number of output samples and the corresponding time elapsed. As shown in Figure 6(a), the base FFT of size 64 gives the highest throughput due to the low computation load and correspondingly has a shorter gap between every iteration. As the FFT size increases, the throughput gradually decreases because AIEs have to handle more computation for each iteration. Besides, the gap between every iteration also grows, which indicates the efficiency of the single AIE decreases with the expansion of the FFT size. One outlier occurs for the FFT size of 128 due to the grouping of data to form 1K inputs, which leads to irregular pointer jumps inside the data rearrangement. The AIE compiler fails to optimize this case and thus assigns abundant cycles to solve it. Other base FFT cases are fully optimized on each single AIE.

B. Single-level Partition Comparison

In this section, we evaluate the performance of single-level partitioning using the 1K-point FFT as a case study, specifically examining partition factors of 2, 4, and 8, corresponding to the configurations $1K = 2 \times 512$, $1K = 4 \times 256$, and

TABLE III: Throughput in Various Multi-level Partition Modes.

Index	Partition Mode	Throughput (MS/s)	AIE_num
1	$1K = 2 \times 2 \times 256$	2986.47	8
2	$1K = 4 \times 2 \times 128$	3183.88	14
3	$1K = 2 \times 4 \times 128$	4290.98	14
4	$1K = 2 \times 2 \times 2 \times 128$	5033.42	20
5	$1K = 2 \times 2 \times 2 \times 2 \times 64$	9226.04	48

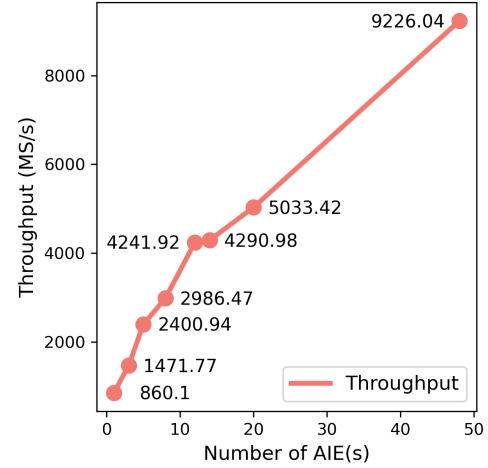


Fig. 7: ESFA Generated Optimal Modes for 1K-point FFT.

$1K = 8 \times 128$. Notably, when the partition factor is set to 8, the second stage requires significantly more processing time than the first stage, resulting in kernel imbalance and pipeline stalling. To address this issue, we further partition the second stage (last layer) into several components to reduce the critical path, which we refer to as the layer partition factor. Figure 6(b) demonstrates the outcomes of this layer partitioning approach aimed at resolving kernel imbalance. We distribute the workload of the second stage across multiple AIEs up to a layer partition factor of 4 to achieve a complete balance among the kernels of different stages. This modified design, with a layer partition factor of 4, demonstrates a performance enhancement of 3.6x compared to the original configuration. In addition to improving throughput, the layer partition also benefits the overall resource efficiency of the design (throughput per core). Consequently, we adopt a layer partition factor of 4 for implementing the $1K = 8 \times 128$ design. Figure 6(c) showcases the performance results of the single-level partition across various partition modes. As the partition factor increases, the system throughput of the 1K-point FFT reaches 4241.92 MS/s, which achieves a 4.9x speedup compared to the single-kernel design. The partition modes 2×512 and 4×256 yield speedups of 1.7x and 2.8x, respectively. Nevertheless, the throughput per core declines progressively as the partition factor increases.

C. Multi-level Partition Comparison

In this section, we apply the ESFA framework to implement the multi-level partition designs of the FFT on the VCK190

board using Vitis 2022.2. Table III reports the performance of five partition modes with randomly selected partition levels and factors. A comparison of partition modes 1, 2, and 3 reveals that, with the same number of partition levels, a larger partition factor enhances system throughput by distributing the computational load across more AIEs. Partition mode 3 demonstrates superior performance compared to mode 2, primarily due to the increased computational time associated with multiplication involving rotation factors at levels with larger partition factors (4 and 8). Consequently, these levels typically exhibit the longest kernel execution time and become the critical stages in the pipeline. By allocating the larger partition factor to the initial level, which corresponds to the closest level to the base FFT, it becomes feasible to evenly distribute the computation load of the rotation factors across multiple base FFTs. Under this configuration, partition mode 3 has a shorter critical path and, therefore, higher throughput than mode 2. Furthermore, an analysis of partition modes 1, 4, and 5 indicates that deeper levels significantly improve total throughput by utilizing more than twice the number of AIEs. Accordingly, the efficiency of each AIE also decreases as the number of levels rises. In addition, the comparison between partition modes 3 and 4 indicates that the advantages of deeper levels outweigh the effects of a larger partition factor, given that the base size FFT is deterministic.

Figure 7 demonstrates the range of optimized partition modes discovered by ESFA when applied to the 1K-point FFT. Through both single-level and multi-level partition approaches, our framework thoroughly explores the design space and identifies more efficient partition modes, enhancing throughput by up to 10.7x compared to the single-kernel design. Except for the performance evaluation, the utilized resources are reported to establish the optimal frontier. This reported optimal frontier aids users in addressing the throughput-resource trade-off based on their specific requirements.

Furthermore, a comprehensive analysis has been conducted to evaluate the performance of our design in comparison to state-of-the-art implementations based on FPGA, ASIC, and CGRA. As demonstrated in Table IV, our proposed multi-level partitioning approach ($1K = 2 \times 2 \times 2 \times 2 \times 64$) achieves an impressive throughput of up to 9226.04 MS/s for 1K-point FFT computations in simulation. Compared to conventional FPGA designs, our approach harnesses the capabilities of the AIE array to realize a performance improvement of 3.4-23.2x. Notably, our data width is twice that of conventional designs, suggesting that the normalized speedups of our design should be higher. Compared to CGRA and ASIC-based designs, our proposed framework demonstrates superior performance and enhanced scalability, supporting FFT sizes from 64 to 32K points. To assess the system performance of designs generated by ESFA, we implemented the 1K-point FFT on the VCK190 platform and obtained the performance according to the run summary. Due to the latency associated with data transfer between DDR-PL and PL-AIE, the final throughput was reduced to 2059.53 MS/s. Nevertheless, our design continues to outperform existing FPGA designs and achieves a performance improvement of 2.2-5.2x.

The design utilizing Xilinx IP [40] and EA4RCA [38]

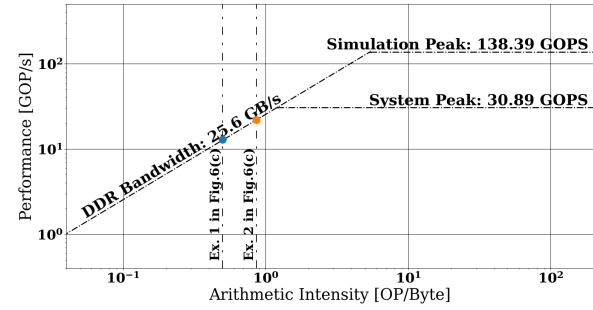


Fig. 8: Roofline Analysis of the ESFA-generated 1K-point FFT.

are the most relevant to our work. As demonstrated in Table IV, our design surpasses the performance of the Xilinx IP design [40], which is dependent on a single AIE, in both simulation and hardware tests. The incorporation of multiple AIEs significantly enhances the overall throughput of the FFT design with a specific size. Furthermore, our single AIE design achieves a speedup of 1.2x compared to the Xilinx IP [40], thereby validating the optimizations implemented in our single kernel approach. In contrast to EA4RCA [38], our design exhibits a speedup of 0.87x; however, it is noteworthy that EA4RCA [38] consumes more resources on both PL and AIEs, as depicted in Table V. Specifically, EA4RCA [38] employs additional DSP resources and utilizes 1.7 times the number of AIEs compared to the design generated by ESFA, resulting in a speedup of 1.14x. Moreover, our framework supports a wider range of FFT sizes and partition modes, thereby providing enhanced flexibility and resource efficiency.

Table V compares the detailed hardware resources utilized by our proposed design and previous works. Conventional FPGA designs are typically created using RTL, which enables precise control of resource utilization. In contrast, although HLS generates our design on the PL part and tends to consume more LUTs and registers, we migrate all computational workload to AIE and free up arithmetic resources such as DSPs on the PL part. Compared to the Xilinx IP design, our speedup scales sublinearly vs. the number of AIEs due to the workload imbalance between kernels and subsequent stalls and the additional latency under the multi-level partition scheme. In comparison to EA4RCA [38], our design utilizes significantly fewer hardware resources, particularly in terms of DSP and AIE. In individual implementations of our proposed design, we observe a normalized speedup of 1.5x per AIE when compared to EA4RCA [38]. Additionally, considering the constraints imposed by the number of AIEs and the design shape, it is theoretically feasible to implement up to five instances of the EA4RCA design [38] and eight instances of the ESFA-generated design within the AIE array, which utilize 400 AIEs and 384 AIEs, respectively. Under this configuration, ESFA-generated design yields an overall speedup of 1.4x in total throughput.

To provide guidance for future optimizations, we conduct a roofline analysis of the performance attained by ESFA. As illustrated in Figure 8, the theoretical peak performance of

TABLE IV: Throughput (MS/s) Comparison between ESFA-generated 1K-point FFT and State-of-the-art Designs.

Work	Size	Platform	Frequency ^a	DW ^b	Simulation ^c	System ^d
MPR-FFT [22]	8K	KU115 FPGA	188MHz	16	-	915.4
Jian [17]	1K	Virtex-6 FPGA	-	16	1220	-
Nguyen [16]	1K	Virtex-7 FPGA	200MHz	16	-	393.85
Ankur [19]	1K	Virtex-7 FPGA	230MHz	16	460	-
Mario [20]	1K	Virtex-7 FPGA	680MHz	16	2720	-
DRRA-2 [41]	1K	CGRA (65nm)	500MHz	16	398.13	-
Shih [24]	1K	ASIC (TSMC 40nm)	526MHz	14	526.32	-
EA4RCA [38]	1K	VCK5000	1.33GHz	32	-	2355.2
Xilinx IP [40]	1K	VCK190	1.25GHz	32	698	593.51
ESFA (Ours)	1K	VCK190	1.25GHz	32	9226.04	2059.43

^a Working frequency. ^b Data bitwidth. ^c Simulation performance. ^d End-to-end hardware test, DDR involved.

TABLE V: Hardware Resource Comparison between ESFA-generated 1K-point FFT and State-of-the-art Designs.

Work	Size	DW ^b	LUT	Register	DSP	Slice	BRAM	AIE
MPR-FFT [22]	8K	16	19923	48877	1097	-	94	-
Jian [17]	1K	16	3671	4359	45	-	10	-
Nguyen [16]	1K	16	-	-	0	15452	0	-
Mario [20]	1K	16	-	-	12	2631	0	-
EA4RCA [38]	1K	32	122650	214782	96	-	562	80
Xilinx IP [40]	1K	32	835	1458	0	242	0	1
ESFA (Ours)	1K	32	31931	12824	0	4145	22	48

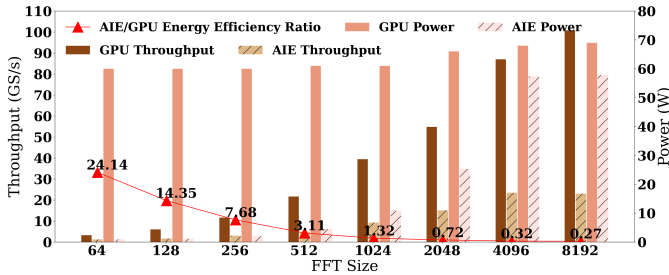


Fig. 9: Energy Efficiency Comparison between ESFA-generated and GPU-based Designs.

ESFA is measured at 138.29 GOPS. However, the limited bandwidth of DDR memory, which is 46 times lower than the bandwidth from PL to AIEs, significantly constrains throughput. Notably, when the design scale is small, the impact of DDR bandwidth is minimal, as demonstrated by the first two examples in Figure 6(c). In light of this, ESFA would benefit from a higher off-chip bandwidth. Enhancements in off-chip bandwidth could yield significant performance improvement, particularly for large-scale designs deployed on the Versal ACAP Platform. Furthermore, the utilization of on-chip buffers may enhance data reuse and mitigate the pressures associated with data transfer.

D. Power Comparison

Furthermore, we conduct a comparative power analysis between our proposed design and the GPU-accelerated FFT. The Nvidia library, implemented on the A100 GPU, serves as the baseline for this evaluation. Each design undergoes 1000 iterations to compute the average throughput. As depicted in Figure 9, the GPU-based method exhibits a higher throughput compared to the design generated by ESFA. However, this advantage is accompanied by significantly elevated power consumption, which results in reduced energy efficiency. In contrast, our proposed solution exhibits a marked improvement in energy efficiency across various FFT sizes, achieving enhancements ranging from 1.32x to 24.14x for FFTs with sizes less than 1k. Consequently, the ESFA framework emerges as a more advantageous option for applications constrained by edge computing and energy limitations, underscoring the robustness and scalability of our design across diverse scenarios. However, as the FFT size increases further, the energy efficiency ratio diminishes due to the superior computational density and memory bandwidth of the GPU. Therefore, a GPU-based design is more appropriate for large-scale data processing.

E. Layout Model Analysis

As previously mentioned, the AIE compiler requires a substantial amount of time to compile a design, making it unsuitable for rapid verification. Our standalone layout model

TABLE VI: Performance Comparison between AIE Compiler & Our Layout Model.

Configuration		Default		Ours	
Index	Partition mode	Time(s)	Involved AIEs	Time(s)	Involved AIEs
1	$1k = 2 \times 512$	130	5	0.18	3
2	$1k = 4 \times 256$	134	8	11	6
3	$1k = 2 \times 2 \times 256$	144	10	13	8
4	$1k = 2 \times 2 \times 2 \times 128$	2021	29	272	23
5	$1k = 2 \times 2 \times 2 \times 2 \times 64$	Fail	Fail	835	85

addresses this issue by reducing the search space through constraints on topology size. Consequently, our layout model is both flexible and lightweight, enabling the rapid identification of feasible solutions for specific designs. The generated layout information can subsequently guide the AIE compiler, thereby decreasing compilation time and improving the success rate. As illustrated in Table VI, our layout model significantly reduces compilation time compared to the AIE compiler, with time reductions ranging from 7.4x to 12.2x. The reduction in time can be attributed to two primary factors. First, the search space is significantly minimized through the pruning of redundant areas relevant to the specific design based on the estimation of the number of AIEs involved. Additionally, the imposition of location constraints (e.g., $C3$ and $C4$) renders certain variable parameters within the layout scheme deterministic, thereby further diminishing the computational effort required. For large-scale design (example 5), our layout model serves as a valuable tool for guiding the AIE compiler towards successful compilation. Furthermore, the design produced by our model utilizes fewer resources than those generated by the AIE compiler, thereby enhancing both area and energy efficiency.

Additionally, our generated layout is more compact, as depicted in Figure 4. This compact layout is suitable for duplication and allows for the instantiation of multiple instances on the AIE array easily. Besides, it is feasible to generate various layout shapes by constraining the topology size in the model, facilitating easier integration with other designs, particularly when resources and space are limited.

F. Analytical Model Analysis

To validate the accuracy of our analytical model, we randomly selected various partition modes for FFT ranging from 1K to 8K. We then compared the estimated throughput derived from our model with the practical throughput obtained through on-board measurements. In this analysis, we excluded the DDR to mitigate the limitations associated with the low bandwidth of off-chip memory, thereby providing a more meaningful evaluation. The AIE operates at a frequency of 1.25 GHz, while the PL side runs at 312.5 MHz. As presented in Table VII, our analytical model demonstrates a high degree of accuracy in estimating throughput across a broad range of FFT sizes, with an average absolute percentage error of less than 3%. Additionally, the number of AIEs is determined by the selected partition modes, allowing our model to consistently report accurate utilization.

TABLE VII: Performance Comparison between Practical Measurements & Analytical Modeling Estimations.

Size	Partition mode	On-board (MS/s)	Estimation (MS/s)	Error (%)
1K	2×512	990.69	1007.31	1.7
1K	$2 \times 2 \times 2 \times 2 \times 64$	6283.17	6492.75	3.2
2K	$2 \times 4 \times 256$	3192.01	3111.11	-2.6
4K	$4 \times 1K$	1666.87	1692.16	1.5
8K	$2 \times 2 \times 2 \times 1K$	3671.17	3756.82	2.3

In summary, our framework facilitates a rapid and resource-efficient end-to-end implementation of FFT tasks on the Versal ACAP. These capabilities effectively address the demands for low latency and real-time performance in FFT-related applications. Additionally, the reduced power consumption aligns well with the needs of edge computing and energy-constrained scenarios. Furthermore, the analytical model offers options for throughput-resource trade-offs, allowing for a balanced strategy to accommodate the diverse demands of various components within FFT-related applications, ultimately leading to an overall optimal solution.

VI. CONCLUSION

This paper presents the ESFA framework, which efficiently implements scalable FFT on the Versal ACAP. It explores optimizations for each base-size FFT on a single AIE and the potential of partitioning an FFT into multiple levels under the guidance of the analytical model and layout model. By partitioning the FFT and leveraging multiple AIEs, our experiments demonstrate speedup of up to 10.1x compared to Xilinx IP and, respectively, 17.5x, 23.2x, and 0.9x speedup compared to the state-of-the-art designs based on ASIC, CGRA, FPGA. Additionally, our design exhibits greater flexibility and resource efficiency compared to other implementations on Versal ACAP platforms.

REFERENCES

- [1] H. Huang and L. Xiao, "Cordic based fast radix-2 dct algorithm," *IEEE Signal Processing Letters*, vol. 20, no. 5, pp. 483–486, 2013.
- [2] M. Garrido, J. Grajal, M. A. Sanchez, and O. Gustafsson, "Pipelined radix-2^k feedforward fft architectures," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 21, no. 1, pp. 23–32, 2013.
- [3] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, pp. 297–301, 1965.
- [4] Z.-G. Ma, X.-B. Yin, and F. Yu, "A novel memory-based fft architecture for real-valued signals based on a radix-2 decimation-in-frequency algorithm," *IEEE Transactions on Circuits and Systems II: Express Briefs (TCAS-II)*, vol. 62, no. 9, pp. 876–880, 2015.
- [5] AMD/Xilinx, "Versal adaptive soc ai engine architecture manual," <https://docs.amd.com/r/en-US/am009-versal-ai-engine/Introduction-to-Versal-Adaptive-SoCs>, 2023.
- [6] R. M. Jiang, "An area-efficient fft architecture for ofdm digital video broadcasting," *IEEE Transactions on Consumer Electronics (TCE)*, vol. 53, no. 4, pp. 1322–1326, 2007.
- [7] C.-P. Fan, M.-S. Lee, and G.-A. Su, "A low multiplier and multiplication costs 256-point fft implementation with simplified radix-24 sdf architecture," in *2006 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, 2006, pp. 1935–1938.
- [8] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM (CACM)*, vol. 21, no. 2, p. 120–126, 1978.
- [9] Nvidia, "Cuda cuFFT," <https://developer.nvidia.com/cufft>, 2023.

- [10] AMD, “clfft,” <https://github.com/clMathLibraries/clFFT>, 2023.
- [11] N. K. Govindaraju, B. Lloyd, Y. Dotsenko, B. Smith, and J. Manferdelli, “High performance discrete fourier transforms on graphics processors,” pp. 1–12, 2008.
- [12] D. B. Lloyd, C. Boyd, and N. Govindaraju, “Fast computation of general fourier transforms on gpus,” in *2008 IEEE International Conference on Multimedia and Expo (ICME)*, 2008, pp. 5–8.
- [13] C. Wang, S. Chandrasekaran, and B. Chapman, “cusfft: A high-performance sparse fast fourier transform algorithm on gpus,” in *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2016, pp. 963–972.
- [14] J. R. Geraci and S. M. Sacco, “A transpose-free in-place simd optimized fft,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 9, no. 3, p. 21, 2012.
- [15] R. Chen, N. Park, and V. K. Prasanna, “High throughput energy efficient parallel fft architecture on fpgas,” in *2013 IEEE High Performance Extreme Computing Conference (HPEC)*, 2013, pp. 1–6.
- [16] N.-H. Nguyen, S. A. Khan, C. H. Kim, and J.-M. Kim, “A high-performance, resource-efficient, reconfigurable parallel-pipelined fft processor for fpga platforms,” *Microprocessors and Microsystems*, vol. 60, pp. 96–106, 2018.
- [17] J. Wang, C. Xiong, K. Zhang, and J. Wei, “A mixed-decimation mdf architecture for radix- 2^k parallel fft,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 24, no. 1, pp. 67–78, 2016.
- [18] M. S. Kavitha and P. Rangarajan, “An efficient fpga architecture for reconfigurable fft processor incorporating an integration of an improved cordic and radix-2r algorithm,” *Circuits, Systems, and Signal Processing (CSSP)*, vol. 39, no. 11, pp. 5801–5829, 2020.
- [19] A. Changela, M. S. Zaveri, and D. Verma, “Fpga implementation of high-performance, resource-efficient radix-16 cordic rotator based fft algorithm,” *Integration*, vol. 73, pp. 89–100, 2020.
- [20] M. Garrido and P. Malagón, “The constant multiplier fft,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 1, pp. 322–335, 2021.
- [21] M. Garrido, J. Grajal, M. A. Sanchez, and O. Gustafsson, “Pipelined radix- 2^k feedforward fft architectures,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 21, no. 1, pp. 23–32, 2013.
- [22] Y. Zhang, L. Zhang, Z. Shang, Y.-R. Su, Z. Wu, and F. Yan, “A new multichannel parallel real-time fft algorithm for a solar radio observation system based on fpga,” *Publications of the Astronomical Society of the Pacific (ASP)*, vol. 134, 2022.
- [23] E. S. Chung, P. A. Milder, J. C. Hoe, and K. Mai, “Single-chip heterogeneous computing: Does the future include custom logic, fpgas, and gpgpus?” pp. 225–236, 2010.
- [24] X.-Y. Shih, H.-R. Chou, and Y.-Q. Liu, “Design and implementation of flexible and reconfigurable sdf-based fft chip architecture with changeable-radix processing elements,” *IEEE Transactions on Circuits and Systems I: Regular Papers (TCAS-I)*, vol. 65, no. 11, pp. 3942–3955, 2018.
- [25] A. X. Glittas, M. Sellathurai, and G. Lakshminarayanan, “A normal i/o order radix-2 fft architecture to process twin data streams for mimo,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 24, no. 6, pp. 2402–2406, 2016.
- [26] M. Wang, F. Wang, S. Wei, and Z. Li, “A pipelined area-efficient and high-speed reconfigurable processor for floating-point fft/fft and dct/iddct computations,” *Microelectronics Journal*, vol. 47, no. C, p. 19–30, 2016.
- [27] J. H. Bahn, J. Yang, and N. Bagherzadeh, “Parallel fft algorithms on network-on-chips,” pp. 1087–1093, 2008.
- [28] S.-s. Zhu, H. Qin, B. Liu, and J. Yang, “Design and optimization of reconfigurable data path for communication baseband signal processing,” pp. 465–4654, 2018.
- [29] Y. Wang, L. Liu, S. Yin, M. Zhu, P. Cao, J. Yang, and S. Wei, “On-chip memory hierarchy in one coarse-grained reconfigurable architecture to compress memory space and to reduce reconfiguration time and data-reference time,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 22, no. 5, pp. 983–994, 2014.
- [30] X. Jia, Y. Zhang, G. Liu, X. Yang, T. Zhang, J. Zheng, D. Xu, H. Wang, R. Zheng, S. Pareek, L. Tian, D. Xie, H. Luo, and Y. Shan, “Xvdp: A high performance cnn accelerator on the versal platform powered by the ai engine,” in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*, 2022, pp. 01–09.
- [31] J. Zhuang, J. Lau, H. Ye, Z. Yang, Y. Du, J. Lo, K. Denolf, S. Neuendorfer, A. Jones, J. Hu, D. Chen, J. Cong, and P. Zhou, “Charm: Composing heterogeneous accelerators for matrix multiply on versal acap architecture,” in *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA)*, 2023, p. 153–164.
- [32] Z. Yang, J. Zhuang, J. Yin, C. Yu, A. K. Jones, and P. Zhou, “Aim: Accelerating arbitrary-precision integer multiplication on heterogeneous reconfigurable computing platform versal acap,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.
- [33] E. Taka, A. Arora, K.-C. Wu, and D. Marculescu, “Maxeva: Maximizing the efficiency of matrix multiplication on versal ai engine,” in *2023 International Conference on Field Programmable Technology (ICFPT)*, 2023, pp. 96–105.
- [34] N. Brown, “Exploring the versal ai engines for accelerating stencil-based atmospheric advection simulation,” in *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA)*, 2023, p. 91–97.
- [35] G. Singh, A. Khodamoradi, K. Denolf, J. Lo, J. Gomez-Luna, J. Melber, A. Bisca, H. Corporaal, and O. Mutlu, “Sparta: Spatial acceleration for efficient and scalable horizontal diffusion weather stencil computation,” in *Proceedings of the 37th ACM International Conference on Supercomputing (ICS)*, 2023, p. 463–476.
- [36] M. Klaisongnoen, N. Brown, T. Dykes, J. R. Jones, and U.-U. Haus, “Evaluating versal ai engines for option price discovery in market risk analysis,” in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA)*, 2024, p. 176–182.
- [37] T. Dai, B. Shi, and G. Luo, “Widesa: A high array utilization mapping scheme for uniform recurrences on acap,” pp. 1–6, 2024.
- [38] W. Zhang, Y. Liu, T. Zang, and Z. Bao, “Ea4rca: Efficient ai accelerator design framework for regular communication-avoiding algorithm,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 21, no. 4, 2024.
- [39] AMD/Xilinx, “Block-by-block configurable fast fourier transform implementation on ai engine,” <https://docs.xilinx.com/r/en-US/xapp1356-fft-ai-engine/Summary>, 2023.
- [40] —, “Versal 2d-fft implementation using vitis acceleration library tutorial,” https://github.com/Xilinx/Vitis-Tutorials/tree/2023.2/AI_Engine_Development/AIE/Design_Tutorials/06-fft2d_AIEvsHLS/AIE, 2023.
- [41] M. A. Shami, M. A. Tajammul, and A. Hemani, “Configurable fft processor using dynamically reconfigurable resource arrays,” *Journal of Signal Processing Systems*, vol. 91, no. 5, p. 459–473, 2019.



Hao Yang (Student Member, IEEE) received the B.S. degree in electronic and communication engineering from City University of Hong Kong, Hong Kong, in 2022, and the M.Phil. degree from the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong, in 2024. He is currently a research assistant with the Department of Electronic and Computer Engineering at the Hong Kong University of Science and Technology, Hong Kong. His current research interests include FPGA-based accelerator design, machine learning, and hardware/software co-design.



Linfeng Du (Student Member, IEEE) received the B.Eng. degree in computer science and technology from the Mixed Class of Chu Kochen Honors College, Zhejiang University, Hangzhou, China, in 2020, and the M.Phil. degree in electronic and computer engineering from The Hong Kong University of Science and Technology, Hong Kong, in 2022. He is currently pursuing the Ph.D. degree with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong.

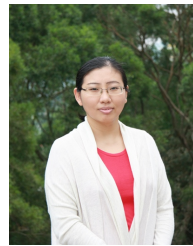
His current research interests include logic and physical optimizations in FPGA design automation, compiler optimizations in high-level synthesis, FPGA-based accelerator design, heterogeneous computing systems, multi-chiplet and SoC-level early modeling, and architectural design optimization.



Shangkun Li received the B.S. degree in physics from Fudan University, Shanghai, China, in 2024.

He is currently a first year Ph.D. student in the Department of Electronic and Computer Engineering at the Hong Kong University of Science and Technology, supervised by Prof. Wei ZHANG.

His current research interests include reconfigurable accelerators and hardware/software co-design.



Wei Zhang (Senior Member, IEEE) received the Ph.D. degree from Princeton University, Princeton, NJ, USA, in 2009.

She was an Assistant Professor with the School of Computer Engineering, Nanyang Technological University, Singapore, from 2010 to 2013. She joined the Hong Kong University of Science and Technology, Hong Kong, in 2013, where she is currently a Professor and she established the Reconfigurable Computing System Laboratory. She has authored or coauthored over 80 book chapters and papers in

peer-reviewed journals and international conferences. Her current research interests include reconfigurable systems, FPGA-based design, low-power high-performance multicore systems, electronic design automation, embedded systems, and emerging technologies.

Dr. Zhang serves as an Associate Editor for ACM Transactions on Embedded Computing Systems, IEEE Transactions on Very Large Scale Integration (VLSI) Systems, and ACM Journal of Emerging Technologies in Computing Systems. She also serves on many organization committees and technical program committees.